RV College of Engineering®

Go, change the world®

UNIT 5
SPARK

Prof.Harshitha V
Dept of AIML
RVCE

Outline

Spark Applications

Jobs

Stages

Tasks

A Scala Standalone Application

Resilient Distributed Datasets

Creation

Transformations and Actions

Persistence

Serialization

Shared Variables

Broadcast Variables, Accumulators

Anatomy of a Spark Job Run

Job Submission, DAG Construction, Task Scheduling, Task Execution

SPARK APPLICATIONS

- **Apache Spark** is an open-source, distributed computing framework used for processing and analyzing large amounts of data quickly.
- It was developed to overcome some of the speed limitations of traditional **Hadoop MapReduce** by processing data **in memory (RAM)** rather than repeatedly reading from and writing to disk.

Key Reasons for Using Spark

- High Speed
- Handles Bigdata
- Supports Real-Time processing
- Machine Learning support
- Fault Tolerance

Spark Applications, Jobs, Stages, and Tasks

- A Spark job is more general than a MapReduce job, though, since it is made up of an arbitrary directed acyclic graph (DAG) of *stages*, each of which is roughly equivalent to a map or reduce phase in MapReduce.
- Stages are split into *tasks* by the Spark runtime and are run in parallel on partitions of an RDD spread across the cluster—just like tasks in MapReduce.
- A job always runs in the context of an *application.*
- An application can run more than one job, in series or in parallel, and provides the mechanism for a job to access an RDD that was cached by a previous job in the same application.

RV College of Engineering®

A Scala Standalone Application

- After working with the Spark shell to refine a program, you may want to package it into a self-contained application that can be run more than once.

```
import org.apache.spark.SparkContext._
import org.apache.spark.{SparkConf, SparkContext}


object MaxTemperature {
  def main(args: Array[String]) {
    val conf = new SparkConf().setAppName("Max Temperature")
    val sc = new SparkContext(conf)

    sc.textFile(args(0))
      .map(_.split("\t"))
      .filter(rec => (rec(1) != "9999" && rec(2).matches("[01459]")))
      .map(rec => (rec(0).toInt, rec(1).toInt))
      .reduceByKey((a, b) => Math.max(a, b))
      .saveAsTextFile(args(1))
  }
}
```

- we use *spark-submit* to run the program, passing as arguments the application JAR containing the compiled Scala program, followed by our program's command-line arguments (the input and output paths):

```
% spark-submit --class MaxTemperature --master local \
  spark-examples.jar input/ncdc/micro-tab/sample.txt output
% cat output/part-*
(1950,22)
(1949,111)
```

RV College of Engineering®

A Java Example

Example 19-2. Java application to find the maximum temperature, using Spark

```
public class MaxTemperatureSpark {

  public static void main(String[] args) throws Exception {
    if (args.length != 2) {
      System.err.println("Usage: MaxTemperatureSpark <input path> <output path>");
      System.exit(-1);
    }

    SparkConf conf = new SparkConf();
    JavaSparkContext sc = new JavaSparkContext("local", "MaxTemperatureSpark", conf);
    JavaRDD<String> lines = sc.textFile(args[0]);
    JavaRDD<String[]> records = lines.map(new Function<String, String[]>() {
      @Override public String[] call(String s) {
        return s.split("\t");
      }
    });
    JavaRDD<String[]> filtered = records.filter(new Function<String[], Boolean>() {
      @Override public Boolean call(String[] rec) {
        return rec[1] != "9999" && rec[2].matches("[01459]");
      }
    });
    JavaPairRDD<Integer, Integer> tuples = filtered.mapToPair(
      new PairFunction<String[], Integer, Integer>() {
        @Override public Tuple2<Integer, Integer> call(String[] rec) {
          return new Tuple2<Integer, Integer>(
              Integer.parseInt(rec[0]), Integer.parseInt(rec[1]));
        }
      }
    );
    JavaPairRDD<Integer, Integer> maxTemps = tuples.reduceByKey(
      new Function2<Integer, Integer, Integer>() {
        @Override public Integer call(Integer i1, Integer i2) {
          return Math.max(i1, i2);
        }
      }
    );
    maxTemps.saveAsTextFile(args[1]);
```

A python Example

Example 19-3. Python application to find the maximum temperature, using PySpark

```
from pyspark import SparkContext
import re, sys

sc = SparkContext("local", "Max Temperature")
sc.textFile(sys.argv[1]) \
  .map(lambda s: s.split("\t")) \
  .filter(lambda rec: (rec[1] != "9999" and re.match("[01459]", rec[2]))) \
  .map(lambda rec: (int(rec[0]), int(rec[1]))) \
  .reduceByKey(max) \
  .saveAsTextFile(sys.argv[2])
```

To run, we specify the Python file rather than the application JAR:

```
% spark-submit --master local \
  ch19-spark/src/main/python/MaxTemperature.py \
  input/ncdc/micro-tab/sample.txt output
```

Spark can also be run with Python in interactive mode using the pyspark command.

Resilient Distributed Datasets

Creation

- There are three ways of creating RDDs: from an in-memory collection of objects using a dataset from external storage (such as HDFS), or transforming an existing RDD.
- The first way is useful for doing CPU-intensive computations on small amounts of input data in parallel.

```
val params = sc.parallelize(1 to 10)
val result = params.map(performExpensiveComputation)
```

- The perform Expensive Computation function is run on input values in parallel.

- The second way to create an RDD is by creating a reference to an external dataset. We have already seen how to create an RDD of String objects for a text file:

  ```
  val text: RDD[String] = sc.textFile(inputPath)
  ```

- Another variant permits text files to be processed as whole files by returning an RDD of string pairs, where the first string is the file path and the second is the file contents.

  ```
  val files: RDD[(String, String)] = sc.wholeTextFiles(inputPath)
  ```

- Spark can work with other file formats besides text. For example, sequence files can be read with:

  ```
  sc.sequenceFile[IntWritable, Text](inputPath)
  ```

Transformations and Actions

- Spark provides two categories of operations on RDDs: ***transformations*** and ***actions.***
- A transformation generates a new RDD from an existing one, while an action triggers a computation on an RDD and does something with the results—either returning them to the user, or saving them to external storage.
- Actions have an immediate effect, but transformations do not—they are lazy, in the sense that they don't perform any work until an action is performed on the transformed RDD.

```
val text = sc.textFile(inputPath)
val lower: RDD[String] = text.map(_.toLowerCase())
lower.foreach(println(_))
```

- One way of telling if an operation is a transformation or an action is by looking at its return type: if the return type is RDD, then it's a transformation; otherwise, it's an action.

Aggregation transformations

- The three main transformations for aggregating RDDs of pairs by their keys are reduceByKey(), foldByKey(), and aggregateByKey().
- reduceByKey(), applies a binary function to values in pairs until a single value is produced.

```
val pairs: RDD[(String, Int)] =
    sc.parallelize(Array(("a", 3), ("a", 1), ("b", 7), ("a", 5)))
val sums: RDD[(String, Int)] = pairs.reduceByKey(_+_)
assert(sums.collect().toSet === Set(("a", 9), ("b", 7)))
```

- we would perform the same operation using foldByKey():

```
val sums: RDD[(String, Int)] = pairs.foldByKey(0)(_+_)
assert(sums.collect().toSet === Set(("a", 9), ("b", 7)))
```

Persistance

- In Spark, **Persistence** is the mechanism of **storing an RDD (or Dataset/DataFrame) in memory or on disk after it is computed**, so that Spark does not have to recompute it every time it is used.
- Calling cache() does not cache the RDD in memory straightaway. Instead, it marks the RDD with a flag indicating it should be cached when the Spark job is run.
- The level is MEMORY_ONLY, which uses the regular in-memory representation of objects.
- A more compact representation can be used by serializing the elements in a partition as a byte array.
- This level is MEMORY_ONLY_SER; it incurs CPU overhead compared to MEMORY_ONLY, but is worth it if the resulting serialized RDD partition fits in memory when the regular in-memory representation doesn't.
- If recomputing a dataset is expensive, then either MEMORY_AND_DISK (spill to disk if the dataset doesn't fit in memory) or MEMORY_AND_DISK_SER (spill to disk if the serialized dataset doesn't fit in memory) is appropriate.

Serialization

- In Spark, **Serialization** is the process of **converting an object into a stream of bytes** so that it can be:
  - Transmitted across a network.
  - Stored on disk.
  - Sent between worker nodes in a cluster.
- There are two aspects of serialization to consider in Spark: **serialization of data** and **serialization of functions.**
- Spark will use Java serialization to send data over the network from one executor to another, or when caching (persisting) data in serialized form.
- Kryo is a more efficient general-purpose serialization library for Java. In order to use Kryo serialization, set the spark.serializer as follows on the SparkConf in your driver program:

```
conf.set("spark.serializer", "org.apache.spark.serializer.KryoSerializer")
```

- Registering classes with Kryo is straightforward. Create a subclass of KryoRegistrator, and override the registerClasses() method:

```
class CustomKryoRegistrator extends KryoRegistrator {
  override def registerClasses(kryo: Kryo) {
    kryo.register(classOf[WeatherRecord])
  }
}
```

Functions

- In Scala, functions are serializable using the standard Java serialization mechanism, which is what Spark uses to send functions to remote executor nodes.
- Spark will serialize functions even when running in local mode, so if you inadvertently introduce a function that is not serializable (such as one converted from a method on a nonserializable class), you will catch it early on in the development process.

Shared Variables

- Spark programs often need to access data that is not part of an RDD.

```
val lookup = Map(1 -> "a", 2 -> "e", 3 -> "i", 4 -> "o", 5 -> "u")
val result = sc.parallelize(Array(2, 1, 3)).map(lookup(_))
assert(result.collect().toSet === Set("a", "e", "i"))
```

Broadcast Variables

- A broadcast variable is serialized and sent to each executor, where it is cached so that later tasks can access it if needed. This is unlike a regular variable that is serialized as part of the closure, which is transmitted over the network once per task.
- Broadcast variables play a similar role to the distributed cache in MapReduce , although the implementation in Spark stores the data in memory, only spilling to disk when memory is exhausted.

```
val lookup: Broadcast[Map[Int, String]] =
    sc.broadcast(Map(1 -> "a", 2 -> "e", 3 -> "i", 4 -> "o", 5 -> "u"))
val result = sc.parallelize(Array(2, 1, 3)).map(lookup.value(_))
assert(result.collect().toSet === Set("a", "e", "i"))
```

Accumulators

- After a job has completed, the accumulator's final value can be retrieved from the driver program.

```
val count: Accumulator[Int] = sc.accumulator(0)
val result = sc.parallelize(Array(1, 2, 3))


  .map(i => { count += 1; i })
  .reduce((x, y) => x + y)
assert(count.value === 3)
assert(result === 6)
```

Anatomy of a Spark Job Run

- At the highest level, there are two independent entities: the *driver*, which hosts the application (SparkContext) and schedules tasks for a job; and the *executors*, which are exclusive to the application, run for the duration of the application, and execute the application's tasks.

Job submission

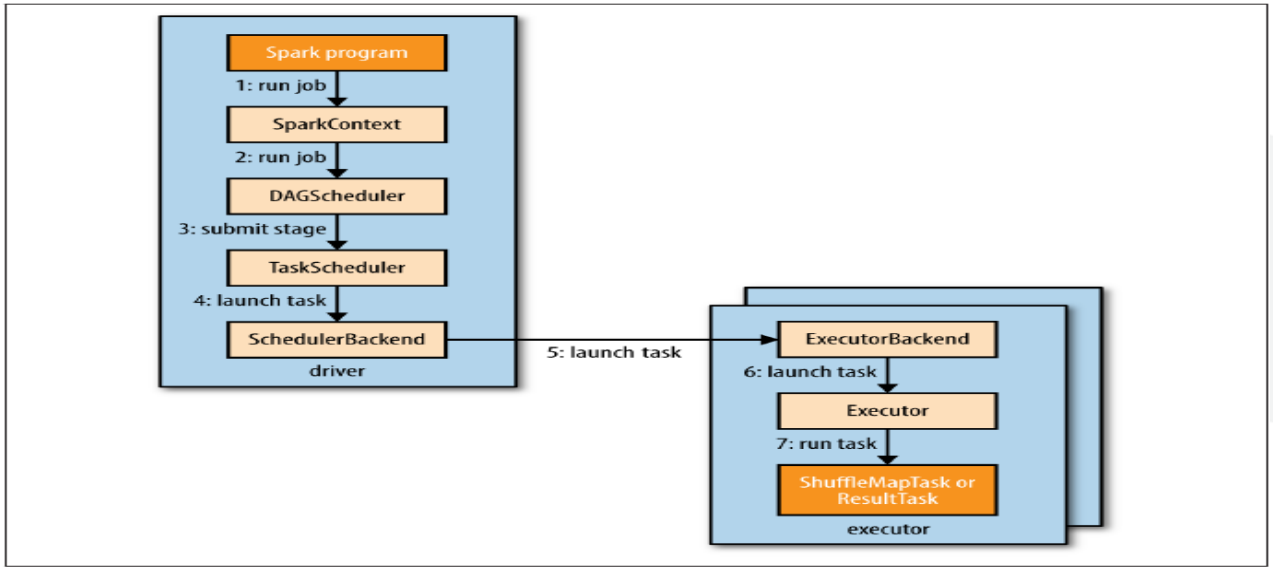

Figure 19-1. How Spark runs a job

- Spark job is submitted automatically when an action (such as count()) is performed on an RDD.
- Internally, this causes runJob() to be called on the SparkContext which passes the call on to the scheduler that runs as a part of the driver .
- The scheduler is made up of two parts: a DAG scheduler that breaks down the job into a DAG of stages, and a task scheduler that is responsible for submitting the tasks from each stage to the cluster.

DAG Construction

- There are two types: *shuffle map tasks* and *result tasks*.
- The name of the task type indicates what Spark does with the task's output:
- ***Shuffle map tasks:*** shuffle map tasks are like the map-side part of the shuffle in MapReduce. Each shuffle map task runs a computation on one RDD partition and, based on a partitioning function, writes its output to a new set of partitions, which are then fetched in a later stage (which could be composed of either shuffle map tasks or result tasks). Shuffle map tasks run in all stages except the final stage.
- ***Result tasks****:* Result tasks run in the final stage that returns the result to the user's program (such as the result of a count()). Each result task runs a computation on its RDD partition, then sends the result back to the driver, and the driver assembles the results from each partition into a final result (which may be Unit, in the case of actions like saveAsTextFile()).

Task Scheduling:

- When the task scheduler is sent a set of tasks, it uses its list of executors that are running for the application and constructs a mapping of tasks to executors that takes placement preferences into account.
- Next, the task scheduler assigns tasks to executors that have free cores, and it continues to assign more tasks as executors finish running tasks, until the task set is complete.
- Executors also send status update messages to the driver when a task has finished or if a task fails. In the latter case, the task scheduler will resubmit the task on another executor.

Task Execution

- First, it makes sure that the JAR and file dependencies for the task are up to date. The executor keeps a local cache of all the dependencies that previous tasks have used, so that it only downloads them when they have changed.
- Second, it deserializes the task code (which includes the user's functions) from the serialized bytes that were sent as a part of the launch task message.
- Third, the task code is executed.
- Tasks can return a result to the driver.
- The result is serialized and sent to the executor backend, and then back to the driver as a status update message.
- A shuffle map task returns information that allows the next stage to retrieve the output partitions, while a result task returns the value of the result for the partition it ran on, which the driver assembles into a final result to return to the user's program.

Thank you!